LAPORAN PRAKTIKUM

POSTTEST (5)

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Alya Kamila Fauziyah (2509106043)

Kelas (A2 ‘25)

PROGRAM STUDI INFORMATIKA

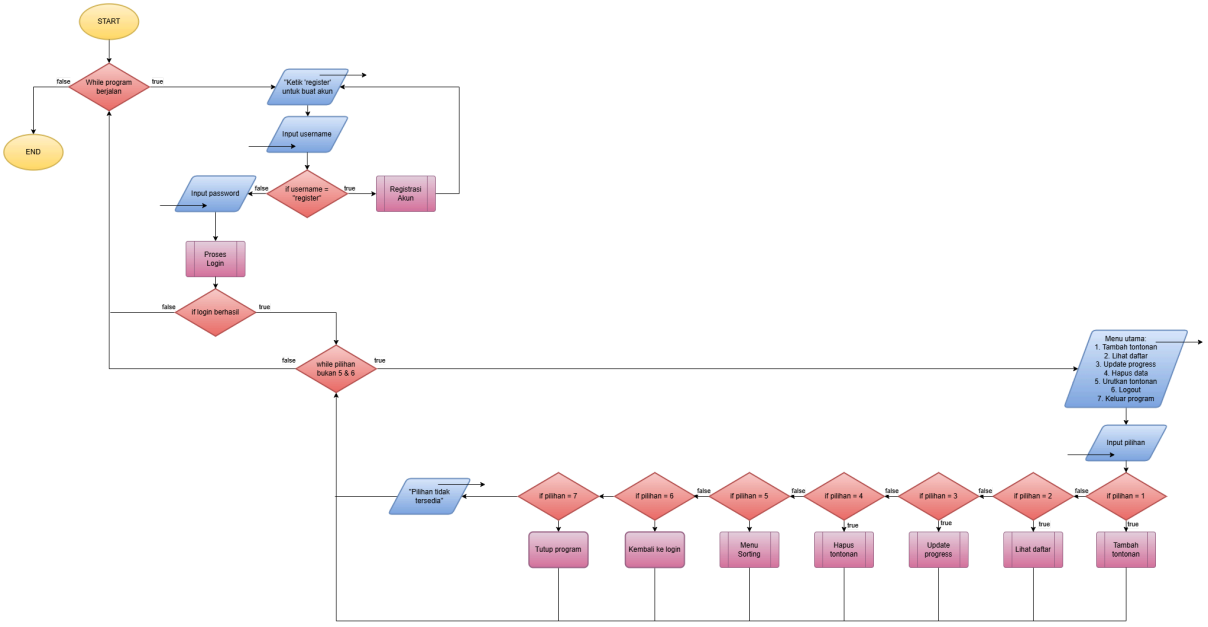
UNIVERSITAS MULAWARMAN

SAMARINDA

2025

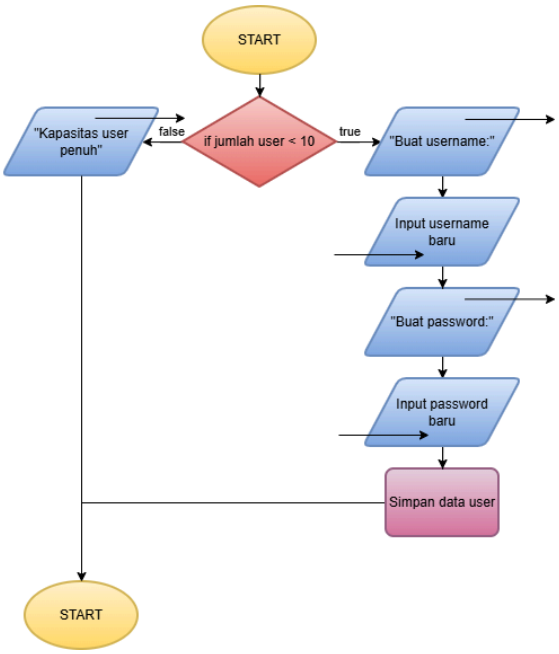
1. Flowchart

1.1 Main



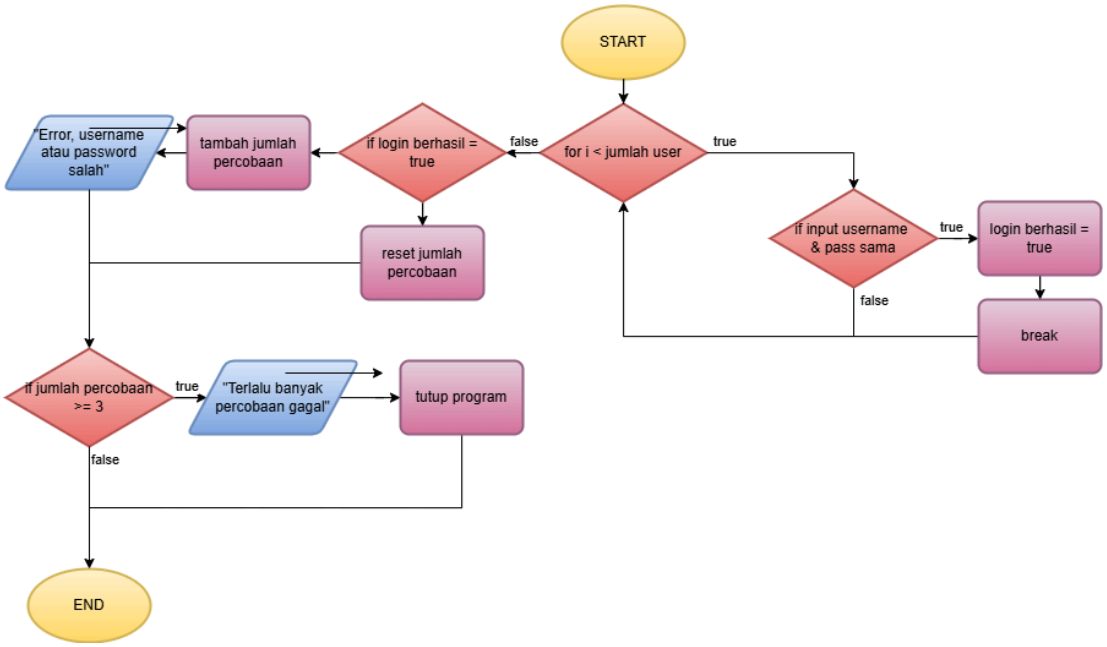
Gambar 1.1 Flowchart Main

1.2 Register



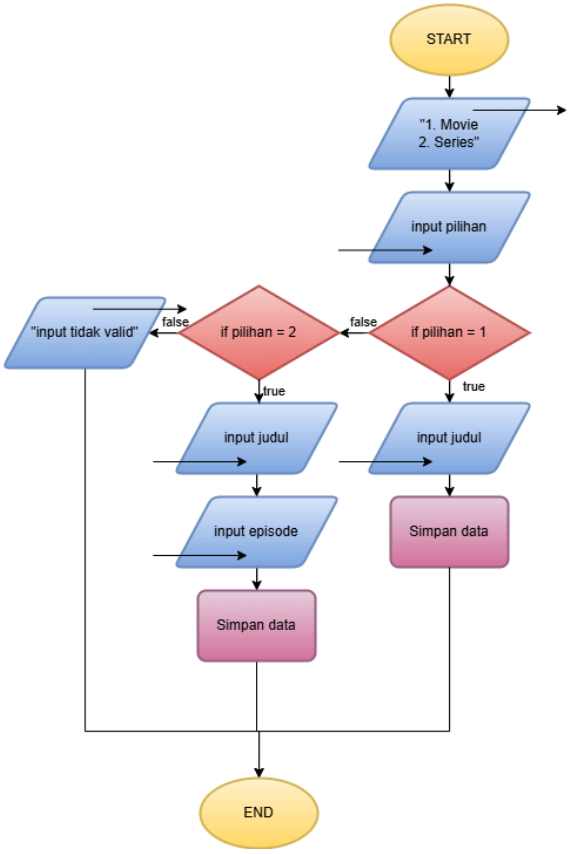
Gambar 1.2 Flowchart Register

1.3 Login



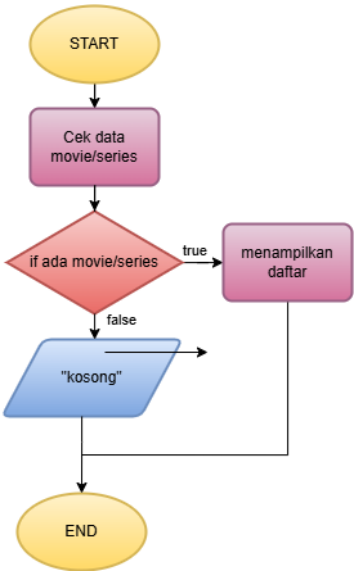
Gambar 1.3 Flowchart Login

1.4 Tambah Tontonan



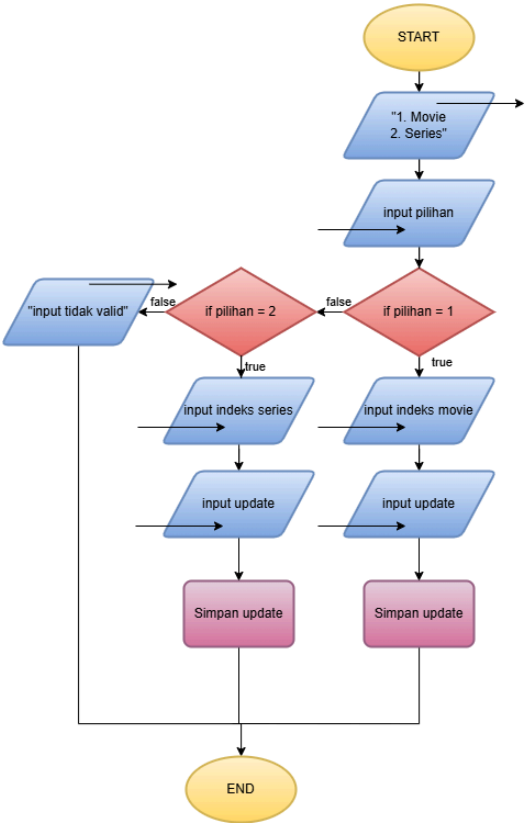
Gambar 1.4 Flowchart Tambah Tontonan

1.5 Lihat Daftar



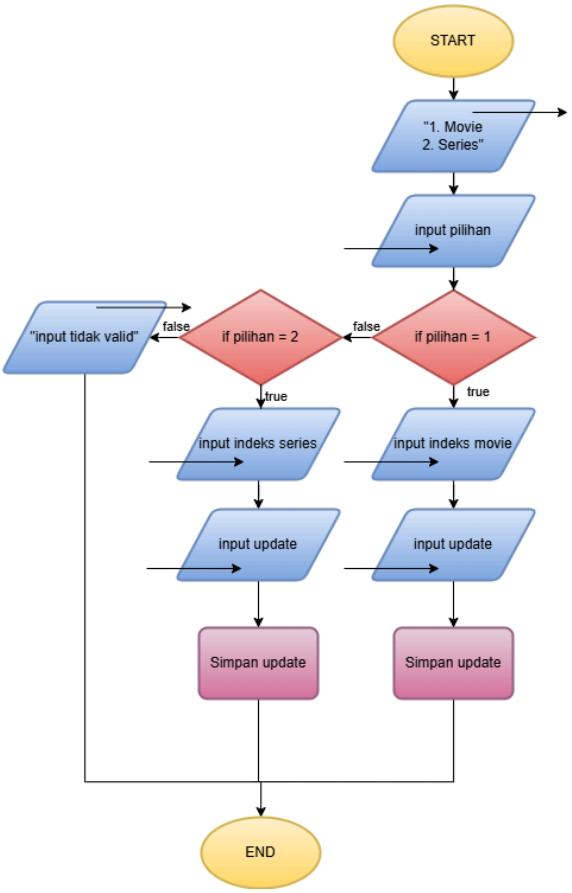
Gambar 1.5 Flowchart Lihat Daftar

1.6 Update Progress



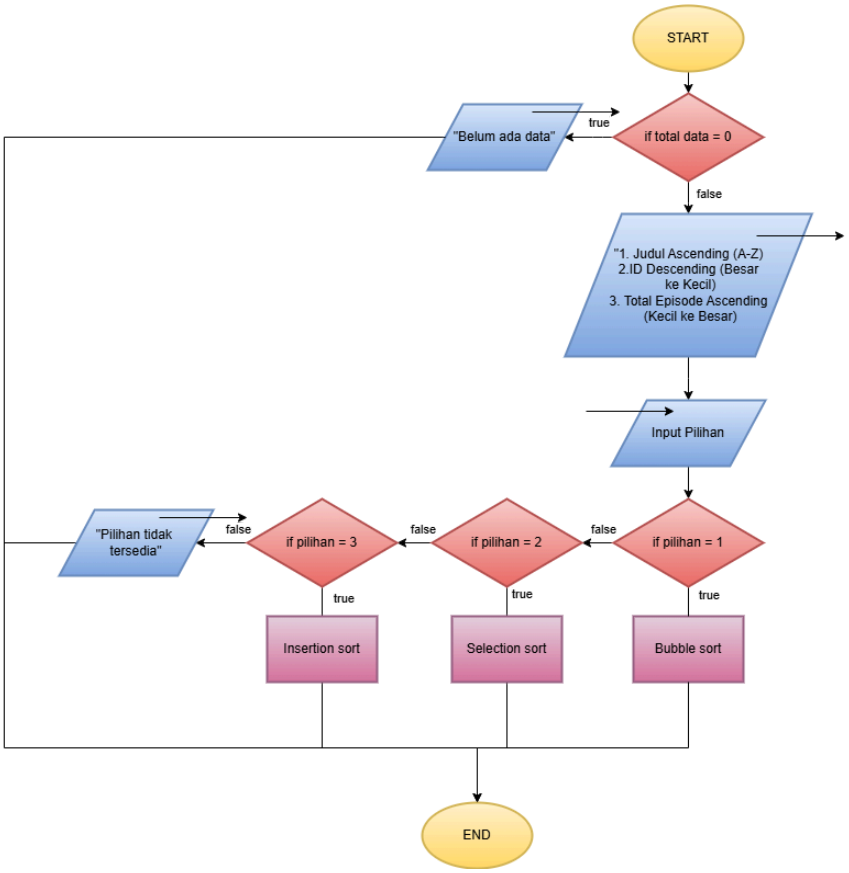
Gambar 1.6 Flowchart Update Progress

1.7 Hapus Data



Gambar 1.7 Flowchart Hapus Data

1.8 Menu Sorting



Gambar 1.7 Flowchart Menu Sorting

2. Deskripsi Singkat Program

Program ini adalah aplikasi berbasis konsol yang berfungsi untuk mencatat dan memantau daftar progres tontonan (Movie dan Series). Program dilengkapi dengan sistem keamanan dan registrasi sederhana berupa fitur Login, di mana pengguna wajib memasukkan username dan password yang benar dengan batas maksimal tiga kali percobaan. Jika login gagal hingga batas percobaan tersebut, program akan otomatis berhenti.

Setelah berhasil masuk, pengguna disuguhkan Menu Utama yang memungkinkan mereka memilih opsi untuk menambah, melihat daftar yang terpisah, memperbarui progress, hingga menghapus data tontonan. Program akan terus berjalan menggunakan perulangan hingga pengguna memilih opsi untuk keluar atau logout. Aplikasi ini dirancang untuk memudahkan manajemen daftar hiburan secara cepat, privat, dan interaktif dalam satu sesi penggunaan.

3. Source Code

3.1 Header, Library, dan Deklarasi Struktur Data (Struct)

```
#include <iostream>
#include <string>
#include <limits>
#include <iomanip>
#include <fstream>
using namespace std;

struct User {
    string username;
    string password;
    int movie_nextid;
    int series_nextid;
};

struct Tontonan {
    int id;
    string judul;
    string tipe;
    int total_eps;
    int eps_ditonton;
    string status;
    User* owner_ptr;
};
```

3.2 Fungsi Utilitas dan Manajemen File (Database)

```
void cetak_garis() { cout << "\n==================================="; }
void cetak_garis(string teks) { cout << "\n=== " << teks << " ==="; }

void swap_tontonan(Tontonan* a, Tontonan* b) {
    Tontonan temp = *a;
    *a = *b;
    *b = temp;
}

const string FILE_NAME = "database_movietrack.txt";

void simpan_data(User* daftar_user, int jumlah_user, Tontonan* daftar,
int jumlah_data) {
    ofstream file(FILE_NAME);
    if (!file.is_open()) {
        cout << "[Peringatan] Gagal membuka file untuk menyimpan
data!\n";
        return;
    }

    file << jumlah_user << "\n";
    for (int i = 0; i < jumlah_user; i++) {
        file << daftar_user[i].username << "\n"
             << daftar_user[i].password << "\n"
             << daftar_user[i].movie_nextid << "\n"
             << daftar_user[i].series_nextid << "\n";
    }

    file << jumlah_data << "\n";
    for (int i = 0; i < jumlah_data; i++) {
        file << daftar[i].id << "\n"
             << daftar[i].judul << "\n"
             << daftar[i].tipe << "\n"
             << daftar[i].total_eps << "\n"
             << daftar[i].eps_ditonton << "\n"
             << daftar[i].status << "\n"
             << daftar[i].owner_ptr->username << "\n";
    }
    file.close();
}

void muat_data(User* daftar_user, int* jumlah_user, Tontonan* daftar,
int* jumlah_data) {
    ifstream file(FILE_NAME);
```

```
    if (!file.is_open()) {
        return;
    }

    file >> *jumlah_user; file.ignore();
    for (int i = 0; i < *jumlah_user; i++) {
        getline(file, daftar_user[i].username);
        getline(file, daftar_user[i].password);
        file >> daftar_user[i].movie_nextid;
        file >> daftar_user[i].series_nextid;
        file.ignore();
    }

    file >> *jumlah_data; file.ignore();
    for (int i = 0; i < *jumlah_data; i++) {
        file >> daftar[i].id; file.ignore();
        getline(file, daftar[i].judul);
        getline(file, daftar[i].tipe);
        file >> daftar[i].total_eps;
        file >> daftar[i].eps_ditonton; file.ignore();
        getline(file, daftar[i].status);

        string owner_username;
        getline(file, owner_username);

        for(int j = 0; j < *jumlah_user; j++) {
            if(daftar_user[j].username == owner_username) {
                daftar[i].owner_ptr = &daftar_user[j];
                break;
            }
        }
    }
    file.close();
}
```

3.3 Algoritma Sorting dan Searching

```
void bubbleSort_JudulAsc(Tontonan* daftar, int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (daftar[j].judul > daftar[j + 1].judul) {
                swap_tontonan(&daftar[j], &daftar[j + 1]);
            }
        }
    }
}

void selectionSort_IdDesc(Tontonan* daftar, int n) {
    for (int i = 0; i < n - 1; i++) {
        int max_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (daftar[j].id > daftar[max_idx].id) {
                max_idx = j;
            }
        }
        if (max_idx != i) {
            swap_tontonan(&daftar[i], &daftar[max_idx]);
        }
    }
}

void insertionSort_EpsAsc(Tontonan* daftar, int n) {
    for (int i = 1; i < n; i++) {
        Tontonan key = daftar[i];
        int j = i - 1;
        while (j >= 0 && daftar[j].total_eps > key.total_eps) {
            daftar[j + 1] = daftar[j];
            j = j - 1;
        }
        daftar[j + 1] = key;
    }
}
```

3.4 Fitur Autentikasi (User Management)

```
void registrasi_akun(User* daftar_user, int* jumlah_user) {
    if (*jumlah_user < 10) {
        cout << "Buat Username: "; cin >>
daftar_user[*jumlah_user].username;
        cout << "Buat Password: "; cin >>
daftar_user[*jumlah_user].password;
        daftar_user[*jumlah_user].movie_nextid = 1;
        daftar_user[*jumlah_user].series_nextid = 1;
        (*jumlah_user)++;
        cout << "Registrasi Berhasil!\n";
    } else {
        cout << "[Error] Kapasitas user penuh!\n";
    }
}

void proses_login(User* daftar_user, int jumlah_user, string user_in,
string pass_in, bool* login_berhasil, int* index_userlogin, int*
jumlah_percobaan, bool* program_berjalan) {
    for (int i = 0; i < jumlah_user; i++) {
        if (user_in == daftar_user[i].username && pass_in ==
daftar_user[i].password) {
            *login_berhasil = true;
            *index_userlogin = i;
            break;
        }
    }

    if (*login_berhasil) {
        *jumlah_percobaan = 0;
    } else {
        (*jumlah_percobaan)++;
        cout << "[Error] Username atau Password salah! Sisa percobaan: "
<< 3 - *jumlah_percobaan << "\n";
        if (*jumlah_percobaan >= 3) {
            cout << "\n[Fatal] Terlalu banyak percobaan gagal. Program
otomatis keluar.\n";
            *program_berjalan = false;
        }
    }
}
```

3.5 Fitur Utama Program (CRUD Data Tontonan)

```
void tambah_tontonan(Tontonan* daftar, int* jumlah_data, User* user_aktif) {
    if (*jumlah_data >= 100) {
        cout << "\n[Pemberitahuan] Maaf, kapasitas daftar sudah penuh!\n";
return;
    }

    cout << "\nTipe (1. Movie, 2. Series, 0. Batal): ";
    int pt; cin >> pt;
    if (cin.fail() || pt < 0 || pt > 2) {
        cin.clear(); cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "[Error] Input tidak valid.\n"; return;
    }
    if (pt == 0) return;

    cout << "Judul: "; cin.ignore(); getline(cin,
daftar[*jumlah_data].judul);
    if (pt == 1) {
        daftar[*jumlah_data].tipe = "Movie";
        daftar[*jumlah_data].id = user_aktif->movie_nextid++;
        daftar[*jumlah_data].total_eps = 1;
    } else {
        daftar[*jumlah_data].tipe = "Series";
        daftar[*jumlah_data].id = user_aktif->series_nextid++;
        cout << "Total Episode: "; cin >> daftar[*jumlah_data].total_eps;

        if (cin.fail() || daftar[*jumlah_data].total_eps <= 0) {
            cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');
            cout << "[Error] Episode tidak valid.\n";
            user_aktif->series_nextid--;
            return;
        }
    }
    daftar[*jumlah_data].eps_ditonton = 0;
    daftar[*jumlah_data].status = "Rencana Menonton";
    daftar[*jumlah_data].owner_ptr = user_aktif;
    (*jumlah_data)++;
    cout << "Data berhasil disimpan!\n";
}

void lihat_daftar(Tontonan* daftar, int jumlah_data, string current_user) {
    cout << "\n>> DAFTAR MOVIE <<\n";
    cout << left << setw(5) << "ID" << setw(55) << "Judul" << "Status\n";

    bool ada_movie = false;
    for (int i = 0; i < jumlah_data; i++) {
```

```
        if (daftar[i].owner_ptr->username == current_user && daftar[i].tipe
== "Movie") {
            cout << left << setw(5) << daftar[i].id
                 << setw(55) << daftar[i].judul
                 << daftar[i].status << endl;
            ada_movie = true;
        }
    }
    if(!ada_movie) cout << "(Kosong)\n";

    cout << "\n>> DAFTAR SERIES <<\n";
    cout << left << setw(5) << "ID" << setw(55) << "Judul" << setw(15) <<
"Progress" << "Status\n";

    bool ada_series = false;
    for (int i = 0; i < jumlah_data; i++) {
        if (daftar[i].owner_ptr->username == current_user && daftar[i].tipe
== "Series") {
            string progress = to_string(daftar[i].eps_ditonton) + "/" +
to_string(daftar[i].total_eps);
            cout << left << setw(5) << daftar[i].id
                 << setw(55) << daftar[i].judul
                 << setw(15) << progress
                 << daftar[i].status << endl;
            ada_series = true;
        }
    }
    if(!ada_series) cout << "(Kosong)\n";
}

void update_progress(Tontonan* daftar, int jumlah_data, string current_user)
{
    cout << "\nUpdate Apa? (1. Movie, 2. Series, 0. Batal): ";
    int up; cin >> up;
    if (cin.fail() || up < 0 || up > 2) {
        cin.clear(); cin.ignore(numeric_limits<streamsize>::max(), '\n');
return;
    }
    if (up == 0) return;

    string tipe_cari = (up == 1) ? "Movie" : "Series";

    cout << "\n--- Daftar " << tipe_cari << " Anda ---";
    cout << "\nID\tJudul\n";
    for (int i = 0; i < jumlah_data; i++) {
        if (daftar[i].owner_ptr->username == current_user && daftar[i].tipe
== tipe_cari) {
            cout << daftar[i].id << "\t" << daftar[i].judul << endl;
```

```
        }
    }

    cout << "\nMasukkan ID (0 Batal): "; int idC; cin >> idC;
    if (cin.fail() || idC == 0) {
        cin.clear(); cin.ignore(numeric_limits<streamsize>::max(), '\n');
return;
    }

    int indeks = cari_id(daftar, jumlah_data, idC, current_user, tipe_cari);

    if (indeks != -1) {
        if (tipe_cari == "Movie") {
            daftar[indeks].eps_ditonton = 1; daftar[indeks].status =
"Selesai";
            cout << "Berhasil diupdate menjadi Selesai!\n";
        } else {
            cout << "Episode terakhir ditonton: "; int eb; cin >> eb;
            if (cin.fail() || eb < 0 || eb > daftar[indeks].total_eps) {
                cin.clear(); cin.ignore(numeric_limits<streamsize>::max(),
'\n');
                cout << "[Error] Input tidak valid.\n";
            } else {
                daftar[indeks].eps_ditonton = eb;
                daftar[indeks].status = (eb == daftar[indeks].total_eps) ?
"Selesai" : "Sedang Ditonton";
                cout << "Berhasil diupdate!\n";
            }
        }
    } else {
        cout << "[Error] ID tidak ditemukan.\n";
    }
}

void hapus_tontonan(Tontonan* daftar, int* jumlah_data, string current_user)
{
    cout << "\nTipe yang dihapus (1. Movie, 2. Series, 0. Batal): "; int th;
cin >> th;
    if (cin.fail() || th < 0 || th > 2) {
        cin.clear(); cin.ignore(numeric_limits<streamsize>::max(), '\n');
return;
    }
    if (th == 0) return;

    string tipe_hapus = (th == 1) ? "Movie" : "Series";
    bool ada_data = false;
    for (int i = 0; i < *jumlah_data; i++) {
        if (daftar[i].owner_ptr->username == current_user && daftar[i].tipe
```

```
== tipe_hapus) ada_data = true;
    }

    if (!ada_data) {
        cout << "(Tidak ada data untuk dihapus)\n"; return;
    }

    cout << "\nMasukkan ID yang ingin dihapus (0 Batal): "; int idH; cin >>
idH;
    if (cin.fail() || idH == 0) {
        cin.clear(); cin.ignore(numeric_limits<streamsize>::max(), '\n');
return;
    }

    int indeks_hapus = cari_id(daftar, *jumlah_data, idH, current_user,
tipe_hapus);

    if (indeks_hapus != -1) {
        char yakin; cout << "Yakin hapus '" << daftar[indeks_hapus].judul <<
"'? (y/n): "; cin >> yakin;
        if (yakin == 'y' || yakin == 'Y') {
            for (int j = indeks_hapus; j < *jumlah_data - 1; j++) {
                daftar[j] = daftar[j + 1];
            }
            (*jumlah_data)--;
            cout << "Data berhasil dihapus!\n";
        }
    } else {
        cout << "[Error] ID tidak ditemukan.\n";
    }
}

void menu_sorting(Tontonan* daftar, int jumlah_data) {
    if (jumlah_data == 0) {
        cout << "\n[Pemberitahuan] Belum ada data untuk diurutkan.\n";
        return;
    }

    cout << "\n=== MENU SORTING ===";
    cout << "\n1. Judul Ascending (A-Z) [Bubble Sort]";
    cout << "\n2. ID Descending (Besar ke Kecil) [Selection Sort]";
    cout << "\n3. Total Episode Ascending (Kecil ke Besar) [Insertion
Sort]";
    cout << "\n0. Batal";
    cout << "\nPilih metode sorting: ";
    int pil; cin >> pil;

    if (cin.fail() || pil < 0 || pil > 3) {
```

```
        cin.clear(); cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "[Error] Pilihan tidak valid.\n"; return;
    }

    if (pil == 1) {
        bubbleSort_JudulAsc(daftar, jumlah_data);
        cout << "Data berhasil diurutkan berdasarkan Judul (A-Z)!\n";
    } else if (pil == 2) {
        selectionSort_IdDesc(daftar, jumlah_data);
        cout << "Data berhasil diurutkan berdasarkan ID (Descending)!\n";
    } else if (pil == 3) {
        insertionSort_EpsAsc(daftar, jumlah_data);
        cout << "Data berhasil diurutkan berdasarkan Total Episode
(Ascending)!\n";
    }
}
```

3.6 Fungsi Utama (int main)

```
int main() {
    Tontonan daftar[100];
    int jumlah_data = 0;
    User daftar_user[10];
    int jumlah_user = 0;
    int jumlah_percobaan = 0;
    bool program_berjalan = true;

    muat_data(daftar_user, &jumlah_user, daftar, &jumlah_data);

    while (program_berjalan) {
        string user_in, pass_in;
        bool login_berhasil = false;
        int index_userlogin = -1;

        cetak_garis();
        cout << "\n   SELAMAT DATANG DI MOVIE-TRACK   ";
        cetak_garis();
        cout << "\n(Ketik 'register' untuk buat akun)";
        cout << "\nUsername: "; cin >> user_in;

        if (user_in == "register") {
            registrasi_akun(daftar_user, &jumlah_user);
            simpan_data(daftar_user, jumlah_user, daftar, jumlah_data);
            continue;
        } else {
            cout << "Password: "; cin >> pass_in;
            proses_login(daftar_user, jumlah_user, user_in, pass_in,
&login_berhasil, &index_userlogin, &jumlah_percobaan, &program_berjalan);
            if (!login_berhasil) continue;
        }

        if (login_berhasil) {
            int pilihan;
            string current_user = daftar_user[index_userlogin].username;
            string judul_menu = "MENU UTAMA (User: " + current_user + ")";

            do {
                cetak_garis(judul_menu);
                cout << "\n1. Tambah Tontonan";
                cout << "\n2. Lihat Daftar (Terpisah)";
                cout << "\n3. Update Progress";
                cout << "\n4. Hapus Data";
                cout << "\n5. Urutkan Tontonan (Sorting)";
                cout << "\n6. LOGOUT";
                cout << "\n7. KELUAR PROGRAM";
```

```
                cout << "\nPilih menu: "; cin >> pilihan;

                if (cin.fail()) {
                    cin.clear();
cin.ignore(numeric_limits<streamsize>::max(), '\n');
                    cout << "\n[Error] Input harus berupa angka!\n";
continue;
                }

                if (pilihan == 1) {
                    tambah_tontonan(daftar, &jumlah_data,
&daftar_user[index_userlogin]);
                    simpan_data(daftar_user, jumlah_user, daftar,
jumlah_data);
                }
                else if (pilihan == 2) lihat_daftar(daftar, jumlah_data,
current_user);
                else if (pilihan == 3) {
                    update_progress(daftar, jumlah_data, current_user);
                    simpan_data(daftar_user, jumlah_user, daftar,
jumlah_data);
                }
                else if (pilihan == 4) {
                    hapus_tontonan(daftar, &jumlah_data, current_user);
                    simpan_data(daftar_user, jumlah_user, daftar,
jumlah_data);
                }
                else if (pilihan == 5) {
                    menu_sorting(daftar, jumlah_data);
                    simpan_data(daftar_user, jumlah_user, daftar,
jumlah_data);
                }
                else if (pilihan == 6) {
                    char yakin; cout << "Yakin Logout? (y/n): "; cin >>
yakin;
                    if (yakin != 'y' && yakin != 'Y') pilihan = 0;
                }
                else if (pilihan == 7) {
                    char yakin; cout << "Yakin Keluar Program? (y/n): "; cin
>> yakin;
                    if (yakin == 'y' || yakin == 'Y') {
                        program_berjalan = false;
                        simpan_data(daftar_user, jumlah_user, daftar,
jumlah_data);
                    }
                    else pilihan = 0;
                }
                else cout << "[Error] Pilihan menu tidak tersedia!\n";
```

```
            } while (pilihan != 6 && pilihan != 7);
        }
    }
    return 0;
}
```

4. Hasil Output

```
==================================
   SELAMAT DATANG DI MOVIE-TRACK
==================================
(Ketik 'register' untuk buat akun)
Username: register
Buat Username: yar
Buat Password: 043
Registrasi Berhasil!
```

Gambar 4.1 Register

```
==================================
   SELAMAT DATANG DI MOVIE-TRACK
==================================
(Ketik 'register' untuk buat akun)
Username: yar
Password: 043
```

Gambar 4.2 Log In

```
=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan (Sorting)
6. LOGOUT
7. KELUAR PROGRAM
```

Gambar 4.3 Menu Utama

```
=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan (Sorting)
6. LOGOUT
7. KELUAR PROGRAM
Pilih menu: 1

Tipe (1. Movie, 2. Series, 0. Batal): 1
Judul: Dead Poet Society
Data berhasil disimpan!
```

Gambar 4.4 Menu 1

```
>> DAFTAR MOVIE <<
ID   Judul                                                Status
5    Avatar: Fire and Ash                                 Rencana Menonton
4    Even if This Love Disappear From the World Tonight   Rencana Menonton
1    Interstellar                                         Rencana Menonton
2    Jojo Rabbit                                          Rencana Menonton
6    Na Willa                                             Rencana Menonton
3    Sonny Boy                                            Rencana Menonton
7    Dead Poet Society                                    Rencana Menonton

>> DAFTAR SERIES <<
ID   Judul                                                Progress      Status
1    Anohimita Hana no Namae wa Bokutachi wa Mada Shiranai 0/12         Rencana Menonton
3    Fumetsu no Anata e                                   0/12          Rencana Menonton
2    Mahou Shoujo Madoka Magica                           0/12          Rencana Menonton
4    Shigatsu wa Kimi no Uso                              0/12          Rencana Menonton

=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan (Sorting)
6. LOGOUT
7. KELUAR PROGRAM
Pilih menu:
```

Gambar 4.5 Menu 2

```
=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan (Sorting)
6. LOGOUT
7. KELUAR PROGRAM
Pilih menu: 3

Update Apa? (1. Movie, 2. Series, 0. Batal): 1

--- Daftar Movie Anda ---
ID      Judul
5       Avatar: Fire and Ash
4       Even if This Love Disappear From the World Tonight
1       Interstellar
2       Jojo Rabbit
6       Na Willa
3       Sonny Boy
7       Dead Poet Society

Masukkan ID (0 Batal): 4
Berhasil diupdate menjadi Selesai!
```

Gambar 4.6 Menu 3

```
=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan (Sorting)
6. LOGOUT
7. KELUAR PROGRAM
Pilih menu: 4

Tipe yang dihapus (1. Movie, 2. Series, 0. Batal): 1

Masukkan ID yang ingin dihapus (0 Batal): 3
Yakin hapus 'Sonny Boy'? (y/n): y
Data berhasil dihapus!
```

Gambar 4.7 Menu 4

```
=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan (Sorting)
6. LOGOUT
7. KELUAR PROGRAM
Pilih menu: 5

=== MENU SORTING ===
1. Judul Ascending (A-Z) [Bubble Sort]
2. ID Descending (Besar ke Kecil) [Selection Sort]
3. Total Episode Ascending (Kecil ke Besar) [Insertion Sort]
0. Batal
Pilih metode sorting:
```

Gambar 4.8 Menu 5

```
=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan (Sorting)
6. LOGOUT
7. KELUAR PROGRAM
Pilih menu: 6
Yakin Logout? (y/n): y

==================================
   SELAMAT DATANG DI MOVIE-TRACK
==================================
(Ketik 'register' untuk buat akun)
Username:
```

Gambar 4.9 Menu 5

```
=== MENU UTAMA (User: yaralyar) ===
1. Tambah Tontonan
2. Lihat Daftar (Terpisah)
3. Update Progress
4. Hapus Data
5. Urutkan Tontonan (Sorting)
6. LOGOUT
7. KELUAR PROGRAM
Pilih menu: 7
Yakin Keluar Program? (y/n): y
PS C:\Praktikum-APL>
```

Gambar 4.10 Menu 6

5. Langkah-langkah GIT

5.1 GIT Add

```
PS C:\Praktikum-APL\Post-test> git add .
```

Gambar 5.1 GIT Add

Perintah git add . digunakan untuk memasukkan semua perubahan file di direktori saat ini ke dalam *staging area*. Proses ini merupakan tahap persiapan sebelum perubahan tersebut direkam secara permanen ke dalam *repository* menggunakan perintah *commit*.

5.2 GIT Commit

```
PS C:\Praktikum-APL> git commit -m "posttest 5"
[main 41248ab] posttest 5
 3 files changed, 550 insertions(+)
 create mode 100644 Post-test/Post-test-APL-5/2509106043-AlyaKamilaFauziyah-PT-5.cpp
 create mode 100644 Post-test/Post-test-APL-5/2509106043-AlyaKamilaFauziyah-PT-5.exe
 create mode 100644 Post-test/Post-test-APL-5/database_movietrack.txt
```

Gambar 5.2 GIT Commit

Perintah git commit -m berfungsi untuk menyimpan semua perubahan dari *staging area* secara permanen ke dalam *repository* lokal. Opsi -m digunakan untuk menyertakan pesan deskriptif singkat secara langsung yang menjelaskan perubahan apa saja yang telah dilakukan pada versi tersebut.

5.3 GIT Push

```
PS C:\Praktikum-APL> git push
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 16 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 102.16 KiB | 9.29 MiB/s, done.
Total 7 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/yaralyar07/praktikum-apl.git
   8efcfe0..41248ab  main -> main
PS C:\Praktikum-APL>
```

Gambar 5.3 GIT Push

Perintah git push berfungsi untuk mengunggah semua perubahan yang sudah di-*commit* dari *repository* lokal ke *repository* jarak jauh (*remote*), seperti GitHub. Langkah ini memastikan versi terbaru dari kodemu tersimpan secara *online* sehingga aman dan dapat diakses dari perangkat lain.